Producer-Worker Task Dispatching System — Technical Documentation

> **문서 버전**: 1.0 **최종 작성일**: 2026-02-22

목차

1. 개요 (Overview)
2. 시스템 아키텍처
3. 주요 구성 요소
    - 3.1 공유 설정 (`config.py`)
    - 3.2 API 서버 (`api.py`)
    - 3.3 Worker (`worker_demo.py`)
    - 3.4 Producer 콘솔 (`producer_demo.py`)
4. Redis 키 스키마
5. 환경 설정 및 실행 방법
6. 예외 처리 및 안정성
7. 데이터 흐름 요약

---

1. 개요 (Overview)

본 시스템은 **Redis 기반의 Producer-Worker 비동기 작업 분배 모델**을 구현합니다. 클라이언트는 FastAPI REST API를 통해 작업을 등록하고, Worker가 해당 큐에서 작업을 꺼내어 처리합니다. 처리 결과는 **작업별 전용 Reply Queue(1:1 응답 채널)** 를 통해 API 서버에 전달되며, 클라이언트에게는 **성공 또는 실패만** 반환됩니다.

1.1 핵심 설계 원칙

항목	설명
동기 대기 응답	API는 작업 등록 후 결과가 나올 때까지 대기하여, 클라이언트에게 성공/실패만 반환합니다.
FIFO 보장	Redis List의 `RPUSH`/`BLPOP` 조합을 통해 선입선출(First-In-First-Out) 순서를 보장합니다.
1:1 결과 전달	작업별 전용 Reply Key(`demo:reply:<job_id>`)를 사용하여, 결과가 정확히 해당 작업의 요청자에게만 전달됩니다.
수평 확장성	Worker를 다수의 프로세스 또는 서버에서 동시에 실행할 수 있으며, Redis의 `BLPOP` 원자적 특성에 의해 하나의 작업은 단 하나의 Worker에게만 할당됩니다.
크래시 내성	Worker의 Python 예외는 try/except로 포착하여 실패 결과를 기록하고, SIGKILL 등 하드 크래시는 API의 BLPOP 타임아웃으로 실패 처리됩니다.
상태 추적	각 작업의 생명주기(`waiting` → `running` → `finish`/`fail`)를 Redis Hash로 실시간 추적합니다.

항목	설명
환경변수 기반 설정	Redis 접속 정보를 `app/config.py`에서 환경변수로 관리하여 로컬/컨테이너 환경 모두 지원합니다.

1.2 Reply Queue 방식의 장점

- **메시지 보장**: Pub/Sub은 구독자가 연결되어 있지 않으면 메시지가 소실되지만, Reply Queue(List + BLPOP)는 소비자가 준비될 때까지 메시지가 보존됩니다.
- **정확한 1:1 매핑**: 작업 ID 기반 전용 키를 사용하므로 결과 혼선이 발생하지 않습니다.
- **타임아웃 제어**: `BLPOP`의 timeout 파라미터를 통해 응답 대기 시간을 유연하게 제어할 수 있습니다.

---

2. 시스템 아키텍처

```
Client (curl / Swagger UI / Locust)
     │  HTTP :8000
     ▼
┌──────────────┐        ┌──────────────┐        ┌────────────────┐
│  api         │──────>│  redis       │<──────│  worker-1      │
│  (FastAPI)   │<──────│  (7-alp.)    │<──────│  worker-2      │
└──────────────┘        └──────────────┘        └────────────────┘
     모두 docker-compose 내부 네트워크
```

처리 순서:

1. 클라이언트가 `POST /jobs`로 작업을 요청합니다.
2. API가 작업 payload를 `demo:job:<id>`에 저장하고, `demo:queue`에 job_id를 `RPUSH`합니다.
3. API가 `demo:reply:<id>`를 `BLPOP`으로 대기합니다 (타임아웃: `work_s + 30초`).
4. Worker가 `demo:queue`를 `BLPOP`으로 대기하다가 job_id를 수신합니다.
5. Worker가 `demo:job:<id>`에서 payload를 읽고 작업을 수행합니다.
6. 작업 완료 후 Worker가 결과를 `demo:reply:<id>`에 `RPUSH`합니다.
7. API가 결과를 수신하여 클라이언트에게 `{"success": true/false}` 형태로 응답합니다.

---

3. 주요 구성 요소

3.1 공유 설정 (`config.py`)

모든 모듈이 공유하는 설정을 환경변수 기반으로 관리합니다.

```python
import os

REDIS_HOST = os.environ.get("REDIS_HOST", "localhost")
REDIS_PORT = int(os.environ.get("REDIS_PORT", "6379"))

QUEUE              = "demo:queue"
JOB_KEY_PREFIX     = "demo:job:"
```

```
STATUS_KEY_PREFIX = "demo:status:"
RESULT_KEY_PREFIX = "demo:result:"
REPLY_KEY_PREFIX  = "demo:reply:"

RESULT_TTL_S        = 3600   # 결과 보관 1시간
REPLY_TTL_S         = 600    # Reply Queue 보관 10분
STATUS_TTL_S        = 3600   # 상태 보관 1시간
JOB_TIMEOUT_GRACE_S = 30     # BLPOP timeout = work_s + 이 값
```

3.2 API 서버 (`api.py`)

FastAPI 기반 REST API로, 작업 등록 후 결과가 나올 때까지 대기하여 **성공/실패만** 반환합니다.

3.2.1 응답 스키마

모든 작업 관련 응답은 동일한 3개 필드를 사용합니다:

```
class JobResponse(BaseModel):
    success: bool    # 성공 여부
    job_id: str      # 작업 ID
    message: str     # 결과 메시지
```

3.2.2 함수 명세

함수	입력	출력	설명
`ts()`	없음	`str`	현재 시각을 `HH:MM:SS.mmm` 형태로 반환합니다.
`get_redis()`	없음	`aioredis.Redis`	Redis 연결 풀을 반환합니다. 연결 없으면 503을 발생시킵니다.
`_wait_for_result(r, job_id, timeout_s)`	Redis, job_id, 타임아웃	`JobResponse`	Reply Key를 BLPOP으로 대기하여 성공/실패 응답을 생성합니다.
`_to_job_response(job_id, result)`	job_id, Worker 결과 dict	`JobResponse`	Worker 결과를 성공/실패 응답으로 변환합니다.
`_mark_fail(r, job_id, detail)`	Redis, job_id, 상세	없음	타임아웃 시 Redis 상태를 fail로 마킹합니다.

3.2.3 `POST /jobs` 동작 흐름

```
async def create_job(req: JobRequest):
    # 1. 8자리 UUID4 기반 고유 작업 ID 생성
    job_id = str(uuid.uuid4())[:8]
```

```
    # 2. payload 저장 + waiting 상태 설정 + FIFO 큐 등록
    await r.set(JOB_KEY_PREFIX + job_id, json.dumps(job))
    await r.hset(STATUS_KEY_PREFIX + job_id, mapping={...})
    await r.rpush(QUEUE, job_id)

    # 3. 결과 대기 (work_s + 30초 타임아웃)
    timeout_s = int(req.work_s) + JOB_TIMEOUT_GRACE_S
    return await _wait_for_result(r, job_id, timeout_s)
    # → {"success": true/false, "job_id": "...", "message": "..."}
```

3.3 Worker (`worker_demo.py`)

Worker는 **동기 방식의 무한 루프 프로세스**로, Redis 큐에서 작업을 하나씩 꺼내어 순차적으로 처리합니다. 크래시 보호를 위해 try/except로 감싸져 있습니다.

3.3.1 모듈 상수

```
WORKER_NAME = socket.gethostname()  # 컨테이너 ID가 워커 식별자로 사용됨
```

TTL 및 키 접두사는 `app.config`에서 임포트합니다.

3.3.2 함수 명세

함수	입력	출력	설명
`new_redis()`	없음	`redis.Redis`	동기 Redis 클라이언트를 생성합니다.
`ts()`	없음	`str`	현재 시각을 `HH:MM:SS.mmm` 형태로 반환합니다.
`set_status(r, job_id, state, detail)`	Redis, job_id, 상태, 상세	없음	작업 상태 Hash를 갱신합니다.
`main()`	없음	없음	메인 루프. BLPOP 대기 → `_process_job` 호출 → 예외 시 실패 기록.
`_process_job(r, job_id)`	Redis, job_id	없음	단일 작업을 처리합니다. 예외 발생 시 main의 except로 전파됩니다.
`_write_result(r, job_id, result)`	Redis, job_id, 결과 dict	없음	결과를 result 키와 reply 키에 기록합니다.

3.3.3 크래시 보호 구조

```
def main():
    while True:
```

```
        _, job_id = r.blpop(QUEUE, timeout=0)
        try:
            _process_job(r, job_id)
        except Exception as e:
            # Python 예외(soft crash) → 실패 결과를 reply key에 기록 → 루프 계속
            result = {"ok": False, "state": "fail", "job_id": job_id, "error":
str(e)}
            r.rpush(REPLY_KEY_PREFIX + job_id, json.dumps(result))
            set_status(r, job_id, "fail", f"worker crash: {e}")
```

크래시 유형	처리 방식
Python 예외 (soft crash)	try/except로 포착 → 실패 결과를 reply key에 기록 → 루프 계속
SIGKILL/OOM (hard kill)	API의 BLPOP 타임아웃(`work_s + 30초`)으로 실패 처리 → 컨테이너 자동 재시작

3.3.4 결과 데이터 형식 (Redis 내부)

성공 시:

```
{
  "ok": true,
  "state": "finish",
  "job_id": "a1b2c3d4",
  "worker": "f7c8e2a1b3d9",
  "took_s": 5.0
}
```

실패 시:

```
{
  "ok": false,
  "state": "fail",
  "job_id": "a1b2c3d4",
  "worker": "f7c8e2a1b3d9",
  "reason": "forced fail"
}
```

3.4 Producer 콘솔 (`producer_demo.py`)

대화형 CLI로, 컨테이너 환경이 아닌 로컬 개발/디버깅 용도로 사용됩니다. `asyncio` 기반으로 작업 등록과 결과 수신을 비동기 병렬 처리합니다.

3.4.1 사용자 명령 체계

입력 형식	동작

입력 형식	동작
`5`	5초 작업을 성공 모드로 등록
`10.5 fail`	10.5초 작업을 실패 모드로 등록
`7 ok`	7초 작업을 성공 모드로 등록
`stats` 또는 `s`	시스템 상태 출력
`help` 또는 `h`	도움말 출력
`quit` 또는 `q`	프로그램 종료

---

4. Redis 키 스키마

키 패턴	타입	TTL	용도
`demo:queue`	List	없음	작업 대기 큐. job_id를 FIFO로 관리합니다.
`demo:job:<id>`	String (JSON)	없음	작업 payload. `job_id`, `work_s`, `fail` 필드를 포함합니다.
`demo:status:<id>`	Hash	3600초	작업 상태. `state`, `worker`, `updated_at`, `detail` 필드를 포함합니다.
`demo:result:<id>`	String (JSON)	3600초	작업 처리 결과. Reply Queue와 별도로 조회할 수 있습니다.
`demo:reply:<id>`	List	600초	1:1 응답 채널. Worker가 `RPUSH`, API가 `BLPOP`으로 사용합니다.

작업 상태 전이도

```
waiting ──> running ──> finish
                    └──> fail
```

상태	설정 주체	설명
`waiting`	API / Producer	큐에 등록된 직후.
`running`	Worker	Worker가 작업을 수신하여 처리 중인 상태.
`finish`	Worker	작업이 정상적으로 완료된 상태.
`fail`	Worker / API	실패 (강제 실패, 크래시, 타임아웃 등).

---

5. 환경 설정 및 실행 방법

5.1 컨테이너 실행 (권장)

```
# 사전 요구: Podman
podman compose up --build -d

# 컨테이너 상태 확인
podman compose ps

# 헬스 체크
curl http://localhost:8000/health
```

서비스 구성:

서비스	컨테이너명	포트	역할
redis	trescal-redis	6379	Redis 7 (Alpine)
api	trescal-api	8000	FastAPI + Uvicorn
worker-1	trescal-worker-1	—	Worker 프로세스
worker-2	trescal-worker-2	—	Worker 프로세스
locust	trescal-locust	8089	Locust 부하 테스트 Web UI

5.2 로컬 실행 (개발용)

```
pip install -r requirements.txt

# 터미널 1: Worker
python -m app.worker_demo

# 터미널 2: API 서버
uvicorn app.api:app --host 0.0.0.0 --port 8000

# 또는 CLI 콘솔
python -m app.producer_demo
```

---

6. 예외 처리 및 안정성

6.1 Worker 측 예외 처리

시나리오	처리 방식
Payload 누락	즉시 `fail` 상태로 전환. 에러 결과를 Reply Queue에 전달.
JSON 파싱 실패	예외가 `_process_job`에서 발생하여 main의 except로 전파. 실패 결과 기록 후 루프 계속.
Python 예외 (soft crash)	try/except로 포착. 실패 결과를 reply key에 기록. 루프 중단 없이 다음 작업 처리.

시나리오	처리 방식
SIGKILL/OOM (hard kill)	컨테이너 `restart: unless-stopped`에 의해 자동 재시작. 처리 중이던 작업은 API의 BLPOP 타임아웃으로 실패 처리.

6.2 API 측 예외 처리

시나리오	처리 방식
BLPOP 타임아웃 (Worker 무응답)	상태를 `fail`로 마킹. `{"success": false, "message": "작업 시간 초과 (Worker 응답 없음)"}` 반환.
BLPOP 중 네트워크 오류	except로 포착. `{"success": false}` 반환.
Redis 연결 실패	`GET /health`에서 503 반환. API healthcheck 실패로 감지.
잘못된 요청 본문	Pydantic 검증에 의해 422 자동 반환.

6.3 컨테이너 자동 재시작

모든 서비스에 `restart: unless-stopped`가 설정되어 있습니다.

상황	재시작 여부
프로세스 크래시 / OOM kill	O
호스트 재부팅 후 Podman 시작	O
`podman compose stop`으로 수동 중지	X

6.4 리소스 자동 정리 (TTL)

키	TTL	근거
`demo:result:<id>`	3600초 (1시간)	결과 조회를 위한 보관 기간
`demo:reply:<id>`	600초 (10분)	1회성 응답 채널
`demo:status:<id>`	3600초 (1시간)	상태 모니터링 보관 기간

---

7. 데이터 흐름 요약

```
클라이언트: POST /jobs {"work_s": 5, "fail": true}
    │
    ▼
┌──────────────────────────────────────────────────────────┐
│  API (api.py)                                            │
│                                                          │
│  1. job_id = uuid4()[:8]                                 │
│  2. SET  demo:job:a1b2c3d4  '{"work_s":5,"fail":true}'   │
│  3. HSET demo:status:a1b2c3d4 state=waiting              │
```

```
│  4. RPUSH demo:queue a1b2c3d4                                  │
│  5. BLPOP demo:reply:a1b2c3d4 (timeout=35초)  ← 대기        │
└──────────────────────────────────────────────────────────────┘
    │  RPUSH
    ▼
┌──────────────┐
│  Redis       │
│  demo:queue  │
└──────────────┘
    │  BLPOP
    ▼
┌──────────────────────────────────────────────────────────────┐
│  Worker (worker_demo.py)                                      │
│                                                               │
│  1. BLPOP demo:queue → "a1b2c3d4"                            │
│  2. GET demo:job:a1b2c3d4 → payload 조회                      │
│  3. HSET demo:status:a1b2c3d4 state=running                  │
│  4. time.sleep(5.0)                                           │
│  5. HSET demo:status:a1b2c3d4 state=fail                     │
│  6. SET  demo:result:a1b2c3d4 '{"ok":false,...}'  (TTL 1h) │
│  7. RPUSH demo:reply:a1b2c3d4 '{"ok":false,...}'  (TTL 10m)│
└──────────────────────────────────────────────────────────────┘
    │  RPUSH
    ▼
┌──────────────────────────────────────────────────────────────┐
│  API (_wait_for_result)                                       │
│                                                               │
│  BLPOP demo:reply:a1b2c3d4 → 결과 수신                       │
│  → {"success": false, "job_id": "a1b2c3d4",                  │
│     "message": "작업 실패: forced fail"}                      │
└──────────────────────────────────────────────────────────────┘
    │
    ▼
  클라이언트 응답 수신
```

> **참고**: 본 시스템은 데모/검증 목적으로 설계되었으며, 작업 수행(`time.sleep`)은 실제 비즈니스 로직으로 대체될 할 예정입니다. 프로덕션 환경 적용 시 Redis 인증 설정, 커넥션 풀링, 구조적 로깅 체계 등을 권장합니다.